



كلية الهندسة المعلوماتية – السنة الرابعة – قسم الذكاء الصناعي

تقرير البرمجة المتوازية

High-Performance E-Commerce Backend Engine

الطلاب المشاركون:

عاطف أسامة شعبان – عمر محمد القاسم

وجد بشار غريب – أمجد علي عيسى – حلا ياسين زين

مقدمة : يهدف هذا المشروع إلى بناء نظام تجارة إلكترونية عالي الأداء قادر على التعامل مع عدد كبير من الطلبات المتزامنة بكفاءة واستقرار، مع تطبيق مفاهيم البرمجة المتوازية والأنظمة الموزعة. يركز المشروع على حماية البيانات من التضارب، إدارة الموارد، تنفيذ المعالجة غير المتزامنة، معالجة البيانات على دفعات، توزيع الأحمال، وضمان سلامة المعاملات والعمليات المتزامنة تحت الضغط العالي. تم تطوير المشروع باستخدام Django و Celery و Threading بهدف بناء نظام مستقر، سريع، وقابل للتوسع.

الطلب الأول: حماية البيانات المشتركة من التضارب (Concurrent Access & Data Integrity)

الحل المستخدم :

القفل التشاؤمي بـ `select_for_update()` داخل كتلة `transaction.atomic()` مدعوماً بـ `Redis.transaction.atomic()`

سبب استخدامه :

منع تداخل المعاملات (Transactions) في نفس اللحظة (جزء من الثانية) عندما يشتري أكثر من مستخدم نفس المنتج.

آلية العمل:

إذا كان المخزون كبيراً، يُخصم من كاش Redis مباشرة. لكن إذا وصل لـ 5 قطع (العتبة الحرجة)، يتدخل `select_for_update()` ليضع قفلاً على صف المنتج في قاعدة بيانات MySQL أي يمكن لأي مستخدم آخر المساس بالمنتج حتى ينهي المستخدم الأول عملية الخصم وتحفظ.

المشكلات التي تم حلها :

مشكلة الـ (Over-selling) ؛ وهي بيع منتج غير موجود أصلاً (مخزون بالسالب) بسبب طلبين تزامنا في نفس الجزء من الثانية.

حل إضافي مستخدم

تم أيضاً استخدام الدالة `F()` :

تسمح بتنفيذ العمليات الحسابية مباشرة داخل قاعدة البيانات بدون الحاجة إلى قراءة القيمة أولاً ثم تعديلها داخل الكود.

الهدف من ذلك:

تنفيذ تحديث مباشر ومن داخل قاعدة البيانات باستخدام `stock_quantity = F('stock_quantity') - 1` بدلا من قراءة القيمة ثم تعديلها يدويا

الفائدة:

- تحديث Atomic وامن
- اسرع تحت الضغط العالي
- تقليل احتمالية حدوث Race Condition
- تحسين الأداء

الطلب الثاني: إدارة الموارد والتحكم بالسعة (Resource Management & Capacity Control)

الحل المستخدم:

الخنق الديناميكي (Dynamic Throttling) والضغط العكسي (Backpressure).

أدوات المستخدمة:

- استخدام redis.Redis.from_url وميثود len('celery') لمعرفة عدد المهام العالقة (Queue Length).
- الوراثة من كلاس UserRateThrottle التابع لمكتبة (Django REST Framework) لعمل خنق ديناميكي للطلبات 1/25s و 3/25s.
- استخدام التوجيه البرمجي بإرجاع status=503 (Service Unavailable) لصد الطلبات الزائدة.

مكانها في الكود :

ملف throttling.py كلاس (CheckoutRateThrottle)، وملف views.py دالة (checkoutLoadDistribution).

سبب استخدامه:

حماية خوادم النظام (CPU و RAM) من الاستهلاك المفرط الذي يؤدي للانهايار.

آلية العمل:

يقوم النظام في CheckoutRateThrottle بقراءة طول طابور المهام في Redis .

إذا زاد الضغط (مثلاً 250 مهمة)، يُقلل معدل قبول الطلبات لـ 3 طلبات لكل 25 ثانية. وإذا بلغ 500، يرفض النظام أي طلب جديد فوراً ويعيد رمز 503 Service Unavailable.

المشكلات التي تم حلها :

التوقف المفاجئ للسيرفر (Server Crash) ، واستنفاد الذاكرة (Out of Memory - OOM) قد تصل مئات الطلبات دفعة واحدة .

- يزداد استهلاك CPU و RAM
- احتمال انهيار السيرفر

اما باستخدامه:

- يبقى السيرفر مستقرًا
- يتم التحكم بالحمل
- تحقيق Capacity Control

الطلب الثالث: المعالجة غير المتزامنة (Asynchronous Processing)

تم استخدام:

- Celery Tasks

- Background Threads

تنفيذ Celery مثل (user.email)(send_verification_email.delay)

شرح الدالة المستخدمة:

delay()

تقوم بإرسال المهمة إلى Celery Queue ليتم تنفيذها بالخلفية بواسطة Worker بدون إيقاف الطلب الرئيسي.

سبب استخدامه:

بدلاً من جعل المستخدم ينتظر إرسال البريد الإلكتروني أثناء تنفيذ الطلب:

- يتم إنهاء الطلب مباشرة

- إرسال المهمة إلى Queue

- يقوم Worker بمعالجتها بالخلفية

الفائدة:

- تقليل زمن الاستجابة

- تحسين تجربة المستخدم

- تقليل الضغط على ال Request الرئيسي

تم استخدام Thread Executor أيضاً من خلال دالة

submit()

تقوم بإرسال مهمة ليتم تنفيذها داخل Thread بالخلفية بشكل غير متزامن.

الهدف:

تشغيل مهام Background محلية باستخدام Threads.

الفائدة:

- تنفيذ المهام بشكل غير متزامن

- عدم تعطيل المستخدم أثناء تنفيذ العمليات الطويلة

- تحسين سرعة النظام

الطلب الرابع: معالجة البيانات على دفعات (Batch Processing)

الحل المستخدم:

تم تنفيذ ذلك داخل `seller_sales_analytics()` حيث تم تقسيم البيانات إلى دفعات صغيرة باستخدام `chunks` ثم معالجة كل دفعة بشكل مستقل.

شرح الدالة المستخدمة:

`seller_sales_analytics()`

تقوم بجلب بيانات المبيعات ثم تقسيمها إلى مجموعات صغيرة وإرسال كل مجموعة لمعالجتها بشكل مستقل.

سبب استخدامه:

بدلاً من معالجة الاف السجلات دفعة واحدة، تم تقسيمها إلى مجموعات صغيرة لتحسين الأداء.

الفائدة:

- تنفيذ Parallel Processing
- سرعة أعلى في المعالجة
- استهلاك ذاكرة أقل
- تحسين قابلية التوسع
- تقليل الضغط على قاعدة البيانات

الطلب الخامس: توزيع الأحمال (Load Distribution)

الدوال المستخدمة:

`group()`

تقوم بتشغيل عدة مهام Celery بالتوازي في نفس الوقت.

`process_checkout_item()`

تقوم بمعالجة منتج واحد داخل الطلب مثل تحديث الكمية وإنشاء `OrderItem`.

`checkoutLoadDistribution()`

تقوم بتقسيم الطلب إلى عدة مهام مستقلة ثم توزيعها على الـ `Workers`.

سبب استخدامه:

يقوم Celery Group بتوزيع المهام على عدة `Workers` بالتوازي.

آلية العمل:

- كل منتج داخل الطلب يتم معالجته بشكل مستقل
- عدة `Workers` يعملون بنفس الوقت

- توزيع الحمل على أكثر من عملية تنفيذ
- الفائدة:

- تسريع تنفيذ الطلبات الكبيرة
- تحسين استجابة النظام
- محاكاة مفهوم Load Balancing
- الاستفادة من المعالجة المتوازية

الطلب السادس: التخزين الموزع (Distributed Caching)

الحل المستخدم:

تم تطبيق الفكرة على الدالة trending_products () وذلك باستخدام:

- Redis Distributed Cache
- Aspect-Oriented Programming (AOP)
- Cache-Aside Pattern

وذلك بهدف تخزين المنتجات الأكثر طلباً وتقليل عدد الاستعلامات المباشرة على قاعدة البيانات.

الدوال المستخدمة:

track_product_view()

تستخدم كـ Decorator لتتبع عدد مرات مشاهدة المنتجات بشكل منفصل عن منطق العمل الأساسي (Business Logic).

trending_products()

تقوم بإرجاع المنتجات الأكثر مشاهدة، مع الاعتماد على Redis أولاً قبل الوصول إلى قاعدة البيانات.

product_details()

تم تطبيق الـ Decorator عليها لتسجيل عدد مرات مشاهدة كل منتج داخل Redis.

آلية العمل:

عند طلب تفاصيل أي منتج عن طريق product_details () يتم تنفيذ decorator :

@track_product_view

يقوم هذا الـ Decorator بزيادة عداد المشاهدات الخاص بالمنتج داخل Redis.

يقوم النظام أولاً بالبحث عن البيانات داخل Redis

إذا كانت البيانات موجودة: Cache Hit

يتم إرجاع النتائج مباشرة من Redis دون الوصول إلى قاعدة البيانات

أما إذا لم تكن موجودة: Cache Miss

يقوم النظام بجلب البيانات من قاعدة البيانات، ثم ترتيب المنتجات حسب عدد المشاهدات وتخزين النتيجة داخل Redis لمدة 300 ثانية (TTL)، وبعدها يتم إرجاع النتيجة للمستخدم.

تم تطبيق مفهوم AOP من خلال فصل منطق تتبع المشاهدات داخل Decorator مستقل عن منطق جلب بيانات المنتج.

الفائدة:

- تقليل عدد الاستعلامات على قاعدة البيانات.
- تحسين سرعة الاستجابة.
- تقليل الحمل على الخادم.
- دعم عدد أكبر من المستخدمين المتزامنين.

الطلب السابع: التحكم في الأقفال (Concurrency Control)

الحل المستخدم:

تم استخدام Pessimistic Locking مع آلية Adaptive Threshold للتحكم بالوصول المتزامن إلى المخزون ومنع تضارب البيانات أثناء عمليات الشراء.

الدالة المستخدمة:

process_checkout_item()

آلية العمل:

يعتمد النظام على Redis لمراقبة كمية المنتج بشكل سريع أثناء تنفيذ الطلبات المتزامنة. عندما تكون كمية المنتج أكبر من 5 يتم استخدام Redis مباشرة لتحديث الكمية بسرعة عالية وتقليل الضغط على قاعدة البيانات، أما عندما تصبح الكمية أقل أو تساوي 5 يتم اعتبار المنتج ضمن حالة Critical Stock ويتم التحويل تلقائياً إلى أسلوب Pessimistic Locking من خلال select_for_update() داخل transaction.atomic() وبذلك يتم قفل سجل المنتج داخل قاعدة البيانات ومنع أي عملية أخرى من تعديله حتى انتهاء العملية الحالية.

الفائدة:

- منع حدوث Race Condition عند محاولة عدة مستخدمين شراء نفس المنتج في الوقت نفسه، خصوصاً عندما تصبح الكمية المتبقية قليلة.
- منع بيع كمية أكبر من المخزون الحقيقي
- حماية المخزون من التعديلات المتزامنة.

- ضمان عدم وصول الكمية إلى قيم سالبة
- تحقيق توازن بين الأداء العالي والحفاظ على سلامة البيانات.
- تطبيق مفهوم Pessimistic Locking بشكل عملي.

الطلب الثامن: سلامة المعاملات (Transaction Integrity / ACID)

الحل المستخدم:

تم استخدام Transaction Atomicity لضمان نجاح جميع العمليات المرتبطة بعملية الشراء أو فشلها بالكامل.

الدوال المستخدمة:

`process_checkout_item()`

`checkoutLoadDistribution()`

آلية العمل:

عند تنفيذ عملية Checkout يتم إنشاء Order جديد ثم توزيع عناصر الطلب على عدة مهام مستقلة لمعالجتها. داخل كل مهمة يتم تنفيذ عمليات تحديث المخزون باستخدام `transaction.atomic()` بحيث تعتبر جميع التعديلات على المنتج جزءاً من معاملة واحدة. في حال حدوث خطأ أثناء تحديث المخزون أو عدم توفر الكمية المطلوبة يتم إلغاء العملية وإرجاع رسالة فشل مناسبة دون تخريب حالة البيانات كما يتم تحديث Redis وقاعدة البيانات بشكل متناسق للحفاظ على سلامة المخزون.

الفائدة:

- ضمان عدم وصول النظام إلى حالة غير متسقة عند حدوث أخطاء أثناء تنفيذ عمليات الشراء المتزامنة.
- الحفاظ على تكامل البيانات.
- تطبيق خاصية Atomicity من خصائص ACID.
- منع إنشاء عمليات شراء غير مكتملة.
- منع فقدان البيانات أثناء الضغط العالي.
- زيادة موثوقية النظام واستقراره تحت الوصول المتزامن.

تحسين آلية معالجة عمليات الشراء

في النسخة الأولية من النظام، تم استخدام آليات محلية للتحكم في التزامن داخل خادم الويب، وذلك من خلال استخدام:

- Semaphore(10) لتقييد عدد عمليات الشراء المتزامنة المسموح بها.
- ThreadPoolExecutor(max_workers=5) لتنفيذ المهام الخلفية بشكل غير متزامن داخل عملية الخادم نفسها.

تم الاعتماد على هذه الآلية في دوال تجريبية مثل buy_product و buy_atomic و buy_f بهدف الحد من الضغط على الخادم أثناء عمليات الشراء واختبار سيناريوهات التنافس على المخزون.

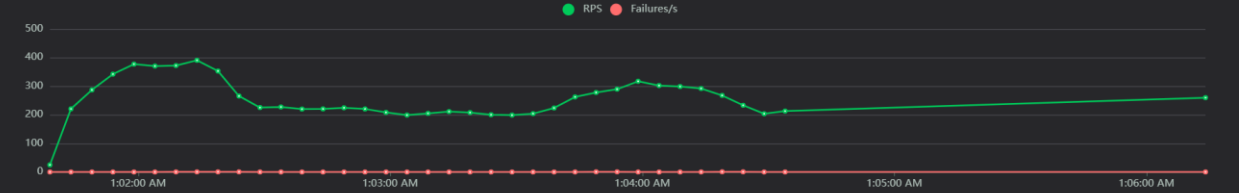
لاحقاً، تم تحسين التصميم واستبدال هذا النهج بآلية توزيع الأحمال باستخدام Celery ، حيث أصبحت عمليات ال Checkout تعتمد على Celery Chord لتوزيع معالجة عناصر الطلب على عدة Workers بشكل متواز، مع استخدام Redis لمراقبة طول قائمة الانتظار وتطبيق سياسة Backpressure عند زيادة الحمل.

نوع الاختناق (Bottleneck)	المظهر والدليل في السجلات (Logs)	التفسير العلمي والتقني للمشكلة	الحل المعماري المطبق
اختناق المعالجة الخلفية (Processing) (Bottleneck)	تراكم المهام في طابور Celery لتصل إلى 832 مهمة معلقة.	تدفق الطلبات المتزامنة من الواجهة الأمامية أسرع بكثير من قدرة العامل الخلفي المتسلسل على معالجتها.	تفعيل خيوط المعالجة المتوازية برمجياً عبر نظام Threads لرفع القدرة الاستيعابية وتجنب الاختناق الزمني.
اختناق منافذ الشبكة (Socket Exhaustion)	ظهور الخطأ الصريح: OperationalError : (2002, 10048	قيام السيرفر بفتح وقفل آلاف اتصالات TCP المتزامنة مع قاعدة البيانات، مما أدى لامتلاء منافذ نظام التشغيل بالكامل.	تطبيق معيار الـ Connection Pool Redis، وتفعيل الاتصالات المستمرة CONN_MAX_AGE MySQL لمنع استهلاك المنافذ.
اختناق الأقفال الصارمة (Database Locking)	تجمد الموقع وظهور أخطاء الـ Timeout عند محاولة الشراء.	استخدام القفل التشاؤمي الصارم على قاعدة البيانات طوال فترة العملية، مما أجبر مئات	Adaptive) (Threshold؛ فحص المخزون الفوري بالـ RAM،

المستخدمين على الانتظار في طابور طويل لفك القفل.	وتفعيل القفل بالـ DB في آخر 5 قطع فقط.		
إطلاق عشرات طلبات الإيميل المتزامنة جعل سيرفر Gmail Spam (يعتبر هذا السلوك هـ Attack) فيقوم بقطع الاتصال قسرياً.	فصل عملية الإيميل في الخلفية كـ Asynchronous Task مستقل مع ميزة الـ retry_backoff المتكاثرة تكتيكياً.	ظهور أخطاء: SMTPServerDisconnected و TimeoutError	اختناق خدمات الطرف الثالث (Third-Party SMTP API)
لو استمر تراكم المهام في Redis لتتخطى عشرات الآلاف، ستفجر الذاكرة العشوائية (RAM) ويحدث انهيار كامل للسيرفر (OOM Crash).	تطبيق نظام Critical Backpressure (Hard Block)؛ قفل البوابة فوراً بكود 503 بمجرد وصول الطابور لـ 500 مهمة لحماية الموارد العتادية.	(تم التنبؤ به وتلافيه برمجياً في الكود لحماية الخادم).	اختناق الذاكرة العشوائية (Potential RAM Exhaustion)

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/token/	100	0	150	270	280	133.06	9	280	494	0	0
POST	/cart/add/	9029	1	390	510	570	362.75	8	1287	57.31	41.4	0
POST	/checkout_distribution/	9003	15	500	680	780	448.61	73	1429	258.36	40.8	0
GET	/product/1/	4022	0	380	510	560	358.67	7	748	9254	16.4	0
GET	/product/2/	4017	0	380	500	560	358.85	7	749	9254	20.3	0
GET	/product/3/	4033	0	380	510	560	358.76	10	740	9254	18	0
POST	/register/	100	0	350	440	500	340.06	166	498	66.92	0	0
GET	/seller_sales?period=day	523	0	360	500	540	344.43	5	600	108	2.4	0
GET	/seller_sales?period=month	506	0	380	510	580	353.94	31	709	108	2.2	0
GET	/seller_sales?period=year	522	0	370	510	560	352.24	18	707	108	2.6	0
GET	/trending_products/	14927	7	380	510	570	355.15	6	1373	220.14	66.7	0
	Aggregated	46782	23	390	600	700	374.86	5	1429	2523.78	210.8	0

Total Requests per Second



Response Times (ms)

